DURGA

SYSTEM AND DEVELOPMENT MANUAL

Frode Randers

2026-06-14

Contents

1 Introduction

Durga is a BPMN-driven Kafka scaffolding tool. It reads BPMN models, extracts an executable subset of process semantics, and generates Java and Kafka-oriented project skeletons intended to be built, reviewed, deployed and operated as normal Kafka applications.

The project is a generator, not a workflow engine:

- BPMN models are parsed with Camunda (but not run in Camunda),
- worker, gateway, orchestrator and event handler classes are rendered with StringTemplate from the process graph,
- Kafka topics and reactive-messaging bindings are generated alongside the code,
- helper scripts and probes are generated so the resulting process can be exercised and observed,
- a separate Kafka Streams monitoring topology materializes process state and operational views.

This manual describes the current system. It focuses on what Durga generates, what runtime model those artifacts implement, and where the support boundary ends. It is written for two audiences: developers who own the generated process code, and platform or DevOps engineers who must provision Kafka topics, deploy workers, monitor process health, and decide which generated artifacts are acceptable for their production environment.

The generated output is not a managed workflow service. Durga supplies source code, topic manifests, helper scripts and monitoring projections; the operating team remains responsible for Kafka durability settings, credentials, deployment topology, alerting, backup/replay policy, and compatibility review before promoting a generated process.

The document is organized in that order. Section 2 describes the major subsystems. Section 4 explains the BPMN-to-Kafka execution mapping. Section 6 describes the canonical process-event and monitoring model. Section 9 describes the generated project output, and Section 25 summarizes maturity, constraints and current gaps.

2 System Overview

Durga has three parts:

- a BPMN parsing and generation pipeline,
- a generated Kafka-oriented process runtime,
- a monitoring and reporting path built around canonical lifecycle events.

2.1 Repository-level components

Scaffolder Parses a BPMN model and renders a generated project (entry point: `org.gautelis.durga.tools.BpmnScaffolder`).
Generation support Model extraction and template coordination: `BpmnModelSupport`, `TaskRoutingGenerator`, `EventTemplateGenerator`, `SubProcessTemplateGenerator`, `GeneratedProjectSupport`.
Templates StringTemplate groups under `src/main/resources/templates/` define generated Java classes, scripts, YAML, Maven build files and shared runtime helpers.
Monitoring runtime A Kafka Streams topology consumes canonical lifecycle events and materializes state, counts, latency summaries and stuck-instance views.

2.2 Generated runtime shape

Each generated project contains:

- worker and routing services,
- a starter,
- a process orchestrator,
- helpers for user/manual task completion and failure testing,
- generated Kafka topic configuration,
- sample payloads and runnable scenario scripts.

Topics and handlers are deliberately visible so that BPMN concepts are projected concretely onto Kafka channels and lifecycle events.

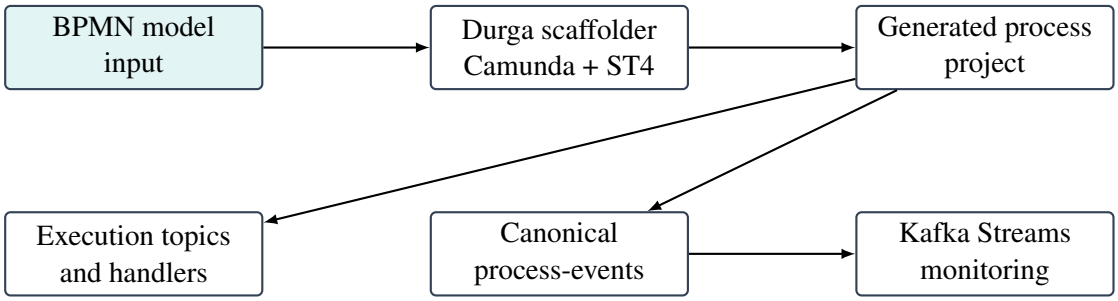

Figure 1: High-level flow from BPMN model to generated runtime and monitoring projections

2.3 Canonical event model

The shared canonical stream on `process-events` is the observability backbone for latest state per instance, counts by current state, latency summaries, stuck-instance detection, dashboards and query endpoints. Execution topics may be useful for routing visibility, but the monitoring model depends on explicit projections rather than raw topic inspection.

3 Architecture Decisions

3.1 Canonical lifecycle stream

The shared `process-events` topic carries normalized lifecycle events. Execution topics carry work between generated handlers, but they are not the authoritative query source. State projections are built from lifecycle events because:

- execution topics represent transport history, not current truth,
- process instance state must be queryable independently of routing topology,
- counts, latency summaries and stuck-instance views are easier to derive from explicit lifecycle events than from task-topic inspection.

3.2 Visible generated runtime

Durga generates many explicit classes instead of a single hidden execution engine. The output is more verbose, but far easier to inspect — making BPMN-to-Kafka mapping concrete.

3.3 Generated helper scripts

Generated probes, scenario runners and manual interaction scripts let users exercise the runtime without writing custom producers and consumers.

3.4 Kafka Streams as the default read model

The monitoring side consumes one canonical stream and materializes queryable state. It solves the local architectural read-model problem directly rather than every production reporting problem.

3.5 Scope-aware subprocess handling

Subprocess support uses explicit entry and completion services because scope boundaries matter when cancellation, failure, escalation and event subprocesses are involved.

4 Execution Model

4.1 General mapping strategy

Durga maps a BPMN graph into a bounded Kafka runtime. In the current implementation, each supported BPMN element is converted into either:

- a generated service consuming from one topic and producing to one or more topics,
- a scope handler that turns subprocess semantics into lifecycle and routing events,
- or an external helper path that lets a human or another tool supply the missing runtime action.

The model is intentionally explicit. Instead of hiding process behavior behind one opaque runtime, the generated project exposes the process topology as concrete Kafka topics, generated services and helper scripts.

4.2 Core control-flow support

The executable subset currently includes:

- start events,
- service, user and manual tasks,
- exclusive and parallel gateways,
- call activities,
- explicit end-event completion,
- embedded subprocesses,
- several categories of boundary events,
- event subprocesses for selected start types.

The important distinction is that not all BPMN elements are treated the same way. A service task auto-completes in the generated worker model, while a user or manual task enters a waiting state and must be advanced through a generated completion helper.

4.3 Tasks and routing

Service tasks are generated as workers consuming `%processId%_%taskId%_input` and producing `%processId%_%taskId%_output`.

User and manual tasks consume their input topic, emit an `ACTIVITY_ENTERED` lifecycle event, and then wait for an external completion event generated via helper scripts.

Exclusive gateways are generated as routing services that evaluate simple conditions against the event payload and emit the selected next activity. Parallel splits fan out to several downstream inputs. Parallel joins use the generated process-state store to wait for the required set of incoming completions before forwarding the flow.

4.4 Boundary behavior

Boundary timer, error and escalation handlers are generated from BPMN boundary events and consume the canonical lifecycle stream on `process-events-monitor` (a logical channel mapped to the physical event topic configured via `-event-topic`). The current boundary implementation supports:

- interrupting and non-interrupting timer boundaries,
- interrupting and non-interrupting error boundaries,
- interrupting and non-interrupting escalation boundaries,
- task-attached and subprocess-attached variants.

Interrupting boundaries now have behavioral effect, not just monitoring effect. Generated handlers consult the shared cancellation registry so that canceled work is suppressed rather than continuing to emit normal downstream progress.

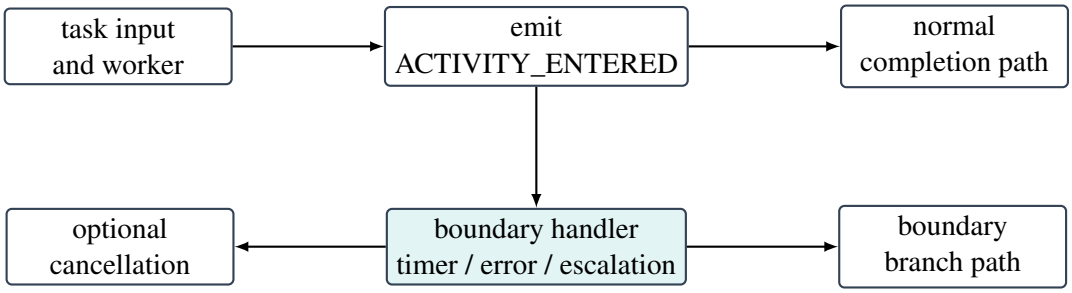

Figure 2: Generated boundary handling separates normal flow from boundary-triggered flow

4.5 Subprocesses and event subprocesses

Embedded subprocesses are generated with explicit entry and completion services. These services emit subprocess-level lifecycle events and route into and out of the internal subprocess graph. Nested subprocesses are supported with the same pattern.

Event subprocesses are currently implemented as scoped handlers rather than as a full general-purpose BPMN scope engine. The supported start types are:

- message,
- signal,
- timer inside an embedded subprocess scope,
- error inside an embedded subprocess scope,
- escalation inside an embedded subprocess scope.

Message and signal event subprocesses consume from dedicated external topics. Timer, error and escalation event subprocesses instead listen to scoped lifecycle signals on `process-events-monitor`. Interrupting event subprocesses emit cancellation for the enclosing scope before starting the event subprocess branch. Non-interrupting event subprocesses branch without canceling the parent flow.

4.6 Current execution boundary

The current execution boundary is broad enough for local process experimentation, but it is not full BPMN coverage. Inclusive gateways, complex gateways, BPMN-native data objects, and more advanced scope semantics are still out of scope. That boundary should be treated as a deliberate implementation limit, not as an accidental omission.

5 BPMN Support And Limits

5.1 Well-supported areas

- start events and explicit end completion,
- service, user, manual, send, receive, script and business rule tasks,
- XOR, inclusive (OR) and parallel gateway routing,
- intermediate timer, message and signal events,
- task and subprocess boundary timer, error and escalation handling,
- call activities,
- embedded subprocesses including nested subprocesses,
- event subprocesses for message, signal, timer, error and escalation starts.

5.2 Current semantic boundaries

- cancellation propagation is pragmatic rather than globally token-precise,
- subprocess scope handling is explicit but lighter than a full BPMN execution scope manager,
- monitoring includes current-state indexes and coarse trend buckets, but is optimized for current-state inspection over long-term analytics,
- generated runtimes prioritize inspectability over minimalism,
- production readiness depends on the generated process implementation and deployment environment, not only on successful scaffolding.

5.3 Outside current scope

- full managed workflow-engine semantics,
- compensation and advanced BPMN exception orchestration,
- turnkey production operation without Kafka, Kubernetes, security and observability review,
- long-term analytics beyond the current monitoring projections.

See `doc/bpmn-kafka-coverage.md` for the per-element support matrix.

6 Monitoring and Observability

6.1 Canonical process-events topic

Durga emits normalized lifecycle events on the shared topic `process-events`. Generated workers, gateways, subprocess handlers, boundary handlers and helper publishers all emit events describing process progress, waiting states, completion, failure, escalation and cancellation.

This separates execution transport from queryable state. Execution topics remain useful for routing, but the monitoring model depends on projections built from explicit lifecycle events. For production use, treat the lifecycle topic as an operational contract: retention, access control, schema compatibility and replay policy determine how far back monitoring can recover and how safely downstream tools can consume process state.

6.2 Kafka Streams topology

A Kafka Streams monitoring topology consumes `process-events` and materializes:

- latest process state by instance,
- counts by current state,
- latency summaries,
- stuck-instance views.

The main entry point is `org.gautelis.durga.monitoring.ProcessMonitoringApp`. The projected instance model is `ProcessStateView`. The query side is exposed through an HTTP service and a CLI client.

6.3 HTTP API

The monitoring service exposes endpoints for health, instance lookup, counts, latency, stuck instances and a dashboard. The local development loop is:

1. generate a process,
2. publish scenarios or task inputs,
3. observe lifecycle events,
4. query current state and latency,
5. iterate on the BPMN model or generated runtime.

In production, the loop is stricter: monitor Streams state, query-store readiness, consumer lag on lifecycle and monitoring topics, stuck-instance counts, latency percentiles, DLQ volume and failed lifecycle events. The dashboard is an operator view over the Kafka Streams read model, not a replacement for Kafka broker, consumer group and infrastructure monitoring.

7 Monitoring Topology

7.1 Purpose

The monitoring subsystem is a separate runtime concern from the generated process execution graph. Generated handlers emit canonical lifecycle events; the monitoring topology consumes those events and turns them into queryable operational views.

Execution topics show how work is routed. The monitoring topology answers:

- where is a specific process instance now,
- how many instances are in a given state,
- which activities are taking the longest,
- which instances appear stuck,
- what are the coarse trend counts for starts, completions, and failures.

7.2 Topology structure

The entry point is `ProcessMonitoringTopology.buildTopology()`. It consumes `process-events`, filters null keys and values, and forks the stream into five parallel sub-topologies, each with a dedicated local state store:

Instance state Aggregates events into `ProcessStateView` per instance. Materialized in `process-state-store`, output to `process-state`. A filtered subset (active instances) is output to `process-active-state`.
State counts Regroups state views by `processId:state` key and counts instances. Materialized in `process-state-counts-store`, output to `process-state-counts`.
Activity latency Feeds events through a custom `ActivityLatencyProcessor` that records `ACTIVITY_ENTERED` timestamps in a persistent key-value store (`process-activity-entry-store`) and, on later completion or termination events, emits duration deltas. These deltas are aggregated into rolling `ActivityLatencyStats` per activity, then mapped to `ActivityLatencySummary` and output to `process-latency`.
Trends Filters to `PROCESS_STARTED`, `PROCESS_COMPLETED`, and `PROCESS_FAILED` events, maps them to hourly trend bucket keys, counts per bucket, and outputs `ProcessTrendPoint` records to `process-trends`. Materialized in `process-trends-store`.
Queryable replicas Five `builder.globalTable()` calls consume the output topics above, each materialized as a global table with a `-global-store` suffix. These are full replicas on every monitoring instance and are what the query service reads.

7.3 State stores

The topology maintains ten state stores:

Store	Type	Purpose
`process-state-store`	Local KTable aggregate	Per-instance latest `ProcessStateView`

Store	Type	Purpose
`process-state-counts-store`	Local KTable count	Counts per `processId:state` key
`process-latency-store`	Local KTable aggregate	Rolling `ActivityLatencyStats` per activity
`process-activity-entry-store`	Persistent KV	Timestamps for the latency temporal join
`process-trends-store`	Local KTable count	Trend bucket counts
`*-global-store` (5×)	Global table	Full replicas for HTTP/CLI query access

The five local stores drive computation — aggregation, counting, and temporal joins. The five global stores serve queries. The output topics (`process-state`, `process-state-counts`, `process-active-state`, `process-latency`, `process-trends`) decouple computation from query access: any monitoring instance can spin up and receive a full read replica without recomputing.

7.4 Latency computation

Activity latency uses a temporal join pattern without windowed stream-stream joins. The `ActivityLatencyProcessor` stores each `ACTIVITY_ENTERED` event's timestamp under the key `instanceId:activityId` in the persistent `process-activity-entry-store`. When a later `ACTIVITY_COMPLETED`, `ACTIVITY_ESCALATED`, `ACTIVITY_CANCELLED`, `GATEWAY_TAKEN`, `PROCESS_COMPLETED`, or `PROCESS_FAILED` event arrives for the same key, the processor computes the duration and emits an `ActivityLatencyDelta`.

These deltas are grouped by a canonical activity key (processId, version, activityId, event type) and aggregated into `ActivityLatencyStats` — a running count, sum, min, and max — then mapped to `ActivityLatencySummary` for output.

7.5 Processing guarantees

The topology is configured with `AT_LEAST_ONCE` semantics. This trades exactly-once guarantees for higher throughput and lower latency. State views may transiently reflect duplicate or stale updates, which is acceptable for a monitoring read model. The state directory defaults to `target/kafka-streams-state` and can be overridden via the `durga.streams.state.dir` system property.

7.6 Query access pattern

The query service (`ProcessMonitoringQueryService`) accesses only the global table stores via `KafkaStreams.store()`. It uses point lookups for instance state and full-store scans for counts, latency, trends, and stuck detection — filtering in memory by process ID where needed. No interactive queries or cross-instance RPC is required, since global tables replicate the entire content to every monitoring instance.

7.7 Relationship to demos

Repository-level demo publishers and scenario runners emit synthetic lifecycle events directly to `process-events`, making the monitoring subsystem independently testable without a full generated process runtime.

7.8 Current shape

The monitoring path is now stronger than the original local-only read model:

- queryable state is replicated from monitoring topics into global read-side stores, so each instance carries a full local copy of the monitoring model,
- latency summaries are maintained incrementally via a custom processor rather than derived from scanning latest-instance state,
- active-instance queries use a dedicated active-state index instead of the full state projection,
- coarse history is maintained as bucketed trend counts for process starts, completions, and failures.

History is intentionally coarse — Durga supports basic trend reporting but is optimized more for current-state inspection than long-retention analytical reporting.

8 Monitoring Interfaces

The monitoring subsystem exposes three interfaces over the same Kafka Streams state:

- an HTTP query service,
- a command-line client,
- a browser dashboard.

8.1 HTTP service

Entry point: `ProcessMonitoringApp`. Query layer: `ProcessMonitoringQueryService` and `ProcessMonitoringHttpServer`.

Endpoints:

- `/health` — Kafka Streams readiness,
- `/instances/{processInstanceId}` — latest instance view,
- `/processes/{processId}/counts` — process-specific state counts,
- `/processes/{processId}/latency` — activity latency summaries,
- `/processes/{processId}/trends` — bucketed start/completion/failure history,
- `/counts` — global state-count view,
- `/stuck` — stuck-instance detection,
- `/dashboard` — browser dashboard.

The HTTP API is designed for internal operational use. Put it behind the same network and identity controls used for other production admin surfaces before exposing it outside a trusted environment.

8.2 CLI client

The repository includes a convenience script at `durga-monitor`:

```
./durga-monitor health
./durga-monitor counts invoice_receipt
./durga-monitor latency invoice_receipt
./durga-monitor stuck invoice_receipt 300
./durga-monitor instance <processInstanceId>
```

Defaults to `http://localhost:8081`. Override with `DURGA_URL`. Equivalent to:

```
java -cp target/durga-0.1.0-beta.1.jar \
  org.gautelis.durga.monitoring.ProcessMonitoringClient \
  http://localhost:8081 <command>
```

8.3 Dashboard

Served from `/dashboard`. Polls the same HTTP endpoints and presents cluster-level process inventory, selected-process state mix, lifecycle trends, latency percentiles, stuck instances and instance details. Built as a standalone Svelte application (`monitoring-ui/`) that compiles to static assets served by the Quarkus backend.

8.4 How the interfaces fit together

All three interfaces consume the same materialized state:

1. events are published to `process-events`,
2. Kafka Streams materializes state and counts,
3. the HTTP layer exposes that materialized state,
4. the CLI and dashboard consume the same HTTP API.

The operator workflow is:

- publish a scenario or task outcome,
- watch `process-events`,
- query one instance or inspect counts and latency,
- use the dashboard for a broader view.

9 Generated Projects

9.1 Generated files

The generated output contains:

- operational classes (workers, gateways, orchestrator) in the configured package,
- contract interfaces for custom activities (*Name*`Contract.java`),
- probe and exerciser classes (starter, publishers, watchers, scenario runner) in `<package>.probes`,
- a copy of the original BPMN model under `src/main/resources/`,
- a generated `pom.xml`,
- merged Kafka channel configuration in `application.yml`,
- a topic-creation script,
- sample payload data in `task-payloads.json`,
- a generated README,
- helper scripts for demos, inputs, completions, failures, escalations and observers.

The BPMN model copy is kept alongside the source code and is the input to regeneration. During the build, `ModelEnricher` writes implementation metadata for custom activities back into this copy, keeping the model in sync with the implementation.

9.2 Contract interfaces and custom workers

When a `ServiceTask` is annotated with `camunda:plugin="custom"`, the scaffolder generates:

- *Name*`Contract.java` — an interface extending `Plugin` that defines the contract for the custom implementation.
- *Name*`WorkerService.java` — a CDI bean that injects the contract interface and delegates each incoming message to the implementation. Includes a null guard and dead-letter routing.

The developer writes a separate implementation class (*Name*`Impl.java`) that implements the contract interface and is annotated with `@ApplicationScoped` for CDI discovery. The implementation class is never touched by the scaffolder.

9.3 Helper scripts

Generated scripts include:

- `demo-scenario.sh`,
- `send-task-input.sh`,
- `complete-task.sh`,
- `fail-task.sh`,
- `escalate-task.sh`,
- `complete-call-activity.sh`,
- `send-message-event.sh`,

- `send-signal-event.sh`,
- `watch-process-events.sh`,
- `watch-task-output.sh`.

9.4 Development workflow

1. Model the pipeline in Camunda Modeler, selecting plugins from the catalog for standard operations and marking custom activities with `plugin="custom"`.
2. Generate the project from the BPMN model with Durga.
3. For custom activities: implement the generated contract interface in a separate Java file. Compile and test with standard tools.
4. Run `ModelEnricher` during the build to write implementation metadata back into the local BPMN model copy.
5. Build the generated project, run a starter or scenario, and observe progress through `process-events` and the monitoring views.
6. When the pipeline needs changes, edit the enriched BPMN model and regenerate. Custom implementations survive untouched; plugin workers are fully regenerated.

10 Deployment

Durga-generated projects are plain Java applications that connect to Kafka. They can be deployed as JARs, Docker containers, or Kubernetes workloads with no Durga-specific runtime service beyond the generated code and a Kafka cluster.

10.1 Architecture

Each generated process instance is a set of Quarkus-based workers (one per BPMN task) communicating through Kafka topics. There is no central orchestrator at runtime — workers consume from input topics and produce to output topics independently. This means:

- every worker is horizontally scalable via Kafka consumer groups,
- worker deployment is stateless; durable state lives in Kafka topics and Kafka Streams stores, while some generated handlers still keep transient in-memory coordination state,
- the generated shaded JAR is self-contained (all dependencies bundled).

10.2 Local development & testing

The recommended local loop uses Docker Compose for Kafka and direct JVM execution for the pipeline:

```
# Start Kafka
cd setup && docker compose up -d

# Build and run the pipeline
cd generated/<project>
START_KAFKA=false BOOTSTRAP=localhost:9094 ./run-local.sh
```

For iterative development, keep Kafka running and rebuild the pipeline:

```
mvn -q clean package -DskipTests
java -cp target/<project>-all.jar \
  org.gautelis.durga.generated.probes.<Process>Starter localhost:9094
```

To test with the monitoring dashboard:

1. Start Kafka as above.
2. Start the monitoring app:

   ```
   cd setup
   START_KAFKA=false ./demo-monitoring.sh
   ```

3. Run pipeline scenarios via the generated helper scripts.
4. Open `http://localhost:18081/dashboard` for the Svelte dashboard or `http://localhost:8081` for the Quarkus REST API.

10.3 Connection to Kafka

Kafka bootstrap servers are configured through a chain of fallbacks:

1. JVM system property `kafka.bootstrap.servers`.
2. Environment variable `KAFKA_BOOTSTRAP_SERVERS`.
3. Hardcoded default `localhost:9094`.

In Docker and Kubernetes, set `KAFKA_BOOTSTRAP_SERVERS` as a container environment variable. The generated `Dockerfile` passes it to the JVM:

```
ENTRYPOINT ["java", "-Xms64m", "-Xmx256m", \
  "-Dkafka.bootstrap.servers=${KAFKA_BOOTSTRAP_SERVERS:-localhost:9094}", \
  "-jar", "/app/pipeline.jar"]
```

10.4 Topic provisioning

Kafka topics must exist before the pipeline starts. There are three approaches, each supported by the generated artifacts.

10.4.1 Pre-creation via shell script

The generated `topics.sh` creates all required topics using the Kafka CLI tools bundled with the Kafka distribution:

```
KAFKA_BOOTSTRAP_SERVERS=kafka-broker:9092 ./topics.sh
```

This is suitable for development, staging, and controlled manual production setups. For production, review partition count, replication factor, cleanup policy and retention before running the script. Automated environments should prefer an init job, a CI/CD provisioning step, or declarative topic management that is owned by the platform team.

10.4.2 Init container

For Kubernetes deployments where the Kafka broker shares the cluster, add an init container to the generated `k8s.yml`:

```
initContainers:
  - name: topic-init
    image: bitnami/kafka:latest
    command: ['/bin/sh', '-c']
    args:
      - |
        /opt/bitnami/kafka/bin/kafka-topics.sh --create \
          --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS \
          --topic <name> --partitions 3 --replication-factor 1
```

10.4.3 Kafka operator (Strimzi)

In Kubernetes environments running the Strimzi operator, topics can be declared as `KafkaTopic` custom resources. A generated `kafka-topics.yml` CRD manifest can be applied alongside the Deployment:

```
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaTopic
metadata:
  name: <process>_<task>_input
  labels:
    strimzi.io/cluster: my-kafka
spec:
  partitions: 3
  replicas: 1
---
```

The scaffolder can generate Strimzi-oriented output when requested. Treat the generated manifest as a starting point: set replicas, partitions, retention and labels to match the target Kafka cluster policy before applying it.

10.5 Test environment deployment

A Docker Compose-based test environment is the fastest way to validate a pipeline end-to-end. The repository's `setup/docker-compose.yml` provides a single-node Kafka broker and Kafka UI.

Add a `docker-compose.test.yml` to the generated project:

```
version: "3"
services:
  pipeline:
    build: .
    environment:
      KAFKA_BOOTSTRAP_SERVERS: kafka:9092
    depends_on:
      - kafka
  kafka:
    image: bitnami/kafka:latest
    environment:
      KAFKA_CFG_NODE_ID: 1
      KAFKA_CFG_PROCESS_ROLES: broker,controller
      KAFKA_CFG_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_CFG_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_CFG_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CFG_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
```

Run with:

```
docker compose -f docker-compose.test.yml up --build
```

10.6 Production deployment

The generated `deploy.sh` supports three targets:

JAR `./deploy.sh` — builds the shaded JAR and runs locally. Suitable for development, VMs and bare-metal deployments when process supervision is provided externally.
Docker `DOCKER=true ./deploy.sh` — builds a Docker image tagged `<processId>:latest`.
Kubernetes `DOCKER=true K8S=true ./deploy.sh` — builds the image and applies the generated `k8s.yml`.

10.6.1 Docker image

The generated `Dockerfile` uses `eclipse-temurin:21-jre-alpine`, runs as a non-root user, and includes a process-based health check. In production, replace or supplement process checks with application-level readiness that verifies Kafka connectivity and worker readiness:

```
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S durga && adduser -S durga -G durga
USER durga
COPY target/*-all.jar /app/pipeline.jar
HEALTHCHECK --interval=30s ...
ENTRYPOINT ["java", "-Xms64m", "-Xmx256m", \
  "-Dkafka.bootstrap.servers=${KAFKA_BOOTSTRAP_SERVERS:-...}", \
  "-jar", "/app/pipeline.jar"]
```

10.6.2 Kubernetes

The generated `k8s.yml` provides:

- A `Deployment` with 2 replicas (configurable), baseline resource requests and limits,
- A `Service` exposing port 8080,
- Exec-based liveness and readiness probes.

10.7 Scaling

Workers scale horizontally via Kafka consumer groups. Adding pods increases concurrency only up to the partition count of the input topic. Scaling therefore requires topic partition planning, idempotent handlers, and a clear policy for external side effects. Three mechanisms are available, from manual to automated.

10.7.1 Manual scaling

```
kubectl scale deployment/<processId>-<taskId> --replicas=4
```

Suitable for development, staging, or controlled incident response. The operator watches consumer lag, processing latency, error rate and downstream service capacity before changing replicas.

10.7.2 KEDA autoscaling (volume-based)

When `-strimzi` or `-separate-workers` is active, the scaffolder generates `keda-scalers.yml` with one `ScaledObject` per worker. Each has dual triggers — Kafka consumer lag as the primary signal, and Prometheus latency as a quality-of-service safety net:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: invoice_receipt-review_invoice
spec:
  scaleTargetRef:
    name: invoice_receipt-review_invoice
  minReplicaCount: 1
  maxReplicaCount: 10
  triggers:
    - type: kafka
      metadata:
        bootstrapServers: kafka-broker:9092
        consumerGroup: invoice_receipt_review_invoice_worker
        topic: invoice_receipt_review_invoice_input
        lagThreshold: "100"
    - type: prometheus
      metadata:
        serverAddress: http://durga-monitoring:8081
        metricName: durga_activity_latency_ms
        threshold: "30000"
        query: durga_activity_latency_ms{pct="p95",
          process_id="invoice_receipt",
          activity_id="review_invoice"}
```

The Kafka trigger scales on message backlog (volume). The Prometheus trigger scales when processing latency degrades despite low lag — for example, when a worker is bottlenecked on an external service or database.

10.7.3 Custom metrics with Prometheus Adapter + HPA (Kubernetes-native)

An alternative for environments without KEDA: Prometheus scrapes `/api/metrics`, the Prometheus Adapter exposes custom metrics to the K8s metrics API, and an HPA scales the Deployment.

```
# ServiceMonitor (prometheus-operator)
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: durga-monitoring
spec:
  selector:
    matchLabels: { app: durga-monitoring }
```

```
  endpoints:
    - path: /api/metrics
      interval: 15s
---
# Prometheus Adapter rule (in its ConfigMap)
- seriesQuery: 'durga_activity_latency_ms{pct="p95"}'
  name:
    matches: "durga_activity_latency_ms"
    as: "activity_latency_p95_ms"
---
# HPA
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: review-invoice-latency
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: invoice_receipt-review_invoice
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Pods
      pods:
        metric:
          name: activity_latency_p95_ms
        target:
          type: AverageValue
          averageValue: "30000"
```

10.7.4 Metrics reference

The monitoring app exposes at `/api/metrics`:

`durga_activity_latency_ms` Gauge. Labels: `pct` (p50, p95, p99, avg), `process_id`, `activity_id`. Value is milliseconds.
`durga_activity_samples` Gauge. Labels: `process_id`, `activity_id`. Total completed samples.
`durga_activity_sla_violations` Counter. Labels: `process_id`, `activity_id`. Count exceeding the configured SLA threshold.
`durga_streams_state` Gauge. 1 if RUNNING, 0 otherwise.

10.8 Monitoring in production

The monitoring topology (`ProcessMonitoringApp`) is a separate deployment that consumes the canonical `process-events` topic and materializes queryable state. In production:

- Deploy the monitoring app as a companion service alongside the pipeline (same Kafka cluster, different consumer group).
- The Quarkus REST API and Svelte dashboard provide operational visibility — instance counts, latency summaries, stuck-instance detection, trend reporting.
- The monitoring topology uses global tables: every instance has a full local replica of the monitoring model. Any instance can answer all queries without cross-instance RPC.
- Protect the REST API and dashboard with network policy, ingress authentication, or an internal-only service boundary. Process instance views may include business keys, correlation ids and error messages.
- Retain the canonical lifecycle event topic for at least the longest monitoring recovery window you expect to support.

10.9 Configuration reference

Variable	Default	Description
`KAFKA_BOOTSTRAP_SERVERS`	`localhost:9094`	Kafka broker list
`HTTP_PORT`	`8080`	Quarkus HTTP port
`PROFILE`	`dev`	Quarkus configuration profile
`START_KAFKA`	`false`	Start Docker Compose Kafka before running
`DOCKER`	`false`	Build Docker image in deploy script
`K8S`	`false`	Apply Kubernetes manifests in deploy script

11 Supervision and Fault Tolerance

Durga runtime components communicate through Kafka topics, which act as durable mailboxes. This architecture enables an Erlang/OTP–like supervision model: workers can crash and restart independently while Kafka preserves unconsumed records according to topic durability and retention settings.

11.1 Architecture

- Every worker consumes from a dedicated Kafka topic (its inbox) and produces to downstream topics.
- Kafka persists messages for a configurable retention period. A crashed worker's unconsumed messages remain in its topic.
- Consumer group rebalancing shifts partitions to surviving instances when a worker pod exits.
- The scaffolder's `-separate-workers` flag generates one Kubernetes Deployment per BPMN task, giving each worker its own fault domain.

11.2 Per-worker isolation

When `-separate-workers` is enabled:

```
./durga src/test/resources/bpmn/invoice_receipt.bpmn --separate-workers
```

Each generated project includes:

- `RegisterInvoiceStandalone.java`, `ReviewInvoiceStandalone.java`, `NotifyRequesterSt` — one standalone worker per BPMN task, each with its own Kafka consumer/producer and poll loop.
- `InvoiceReceiptWorkerBootstrap.java` — entry point that reads the `WORKER_ID` environment variable and starts the matching worker.
- `Dockerfile.<task>` — per-worker Docker image building from the same JAR, with `WORKER_ID` pre-set.
- `k8s.<task>.yml` — per-worker Kubernetes Deployment with isolated resource limits, health probes, and `WORKER_ID` environment variable.

11.3 Supervision policies

Policy	Implementation
Crash recovery	Kafka consumer offset tracking. Worker restarts replay from last committed offset.
Fault isolation	Per-worker Deployments. One worker's OOM does not affect others.
Health signals	Exec-based liveness/readiness probes per pod (check process alive). Extendable to application-level health via `quarkus-smallrye-health`.
Scaling	`kubectl scale` per Deployment. Bounded by topic partition count.
Dead letter	Plugin executor routes failures to `%task%_dlq` topic. Standalone workers log errors and continue.

Backpressure	Kafka consumer group throttles ingestion. Consumer lag available via JMX metrics.

11.4 Recovery guarantees

Message durability Messages are not lost on worker crash. Kafka retains uncommitted records according to the topic's replication, acknowledgement and retention configuration.

At-most-once vs at-least-once Standalone workers commit after processing (`commitSync`). The poll loop catches and logs errors without crashing. Plugin-generated executors route failures to the DLQ. The architecture provides at-least-once delivery with idempotent output and external side-effect handling being the application owner's responsibility.

Process continuity If the starter crashes, the process never starts (one-shot job, retried by K8s). If a worker crashes, Kafka consumer group rebalancing reassigns its partitions. If the final orchestrator crashes, the process remains in-flight until a new orchestrator pod picks up.

11.5 Limitations

- Worker state is external (Kafka). In-memory state such as running aggregations or cancellation registrations is lost on crash unless that specific generated handler persists it.
- The `-separate-workers` model generates poll-loop workers rather than Quarkus CDI beans. These trade CDI features (dependency injection, reactive messaging) for isolated fault domains and smaller per-pod memory footprints.
- Cross-worker coordination (AND joins, subprocess completion) relies on the `ProcessStateStore`, which is itself a Kafka consumer. Its crash characteristics are the same as any other worker.
- Kafka durability does not imply business-level exactly-once semantics. Handlers that call databases, HTTP services or file systems must be idempotent or protected by application-level deduplication.

12 Kafka Connect Integration

Kafka Connect handles data acquisition. Durga owns the topic topology; Connect owns the I/O to external systems. The scaffolder's `-connect` flag generates ready-to-deploy connector configurations for the pipeline's intake and output topics.

12.1 Architecture

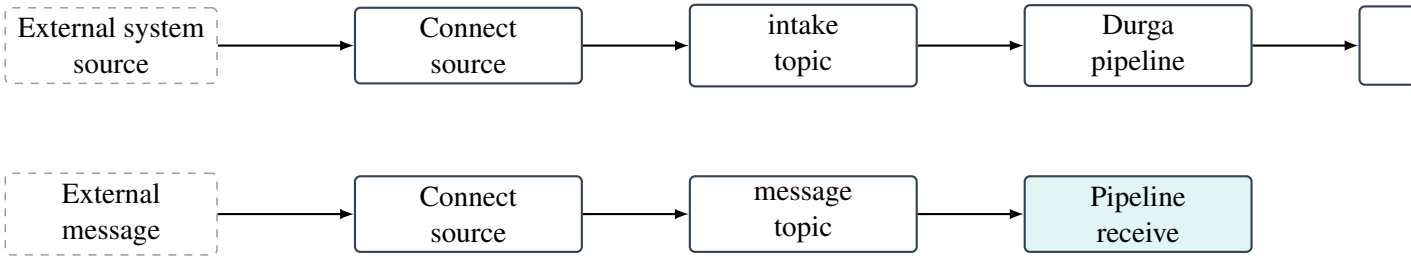

Figure 3: Kafka Connect integration: sources feed intake topics, sinks consume output topics

12.2 Enabling

```
./durga path/to/process.bpmn --connect
```

This adds a `connect/` directory to the generated project:

```
connect/connect-source.json
connect/connect-sink.json
connect.sh
```

12.3 Source connectors

`connect-source.json` contains connector configuration for:

- the process start topic (`%processId%_start`),
- message catch and receive task topics.

The user fills in the connector class and system-specific settings:

```
{
  "name": "invoice_receipt-source",
  "config": {
    "connector.class": "...",
    "tasks.max": "1",
    "invoice_receipt_start": {
      "kafka.topic": "invoice_receipt_start"
    }
  }
}
```

12.4 Sink connectors

`connect-sink.json` contains connector configuration for:

- terminal task outputs,
- the canonical `process-events` topic (for auditing or replication),
- message throw and send task topics.

12.5 Deploying

The generated `connect.sh` posts both configs to the Connect REST API:

```
CONNECT_URL=http://kafka-connect:8083 ./connect.sh
```

Connector status can be checked with:

```
curl http://kafka-connect:8083/connectors/invoice_receipt-source/status
curl http://kafka-connect:8083/connectors/invoice_receipt-sink/status
```

12.6 Common connector classes

Connector	Use case
`DebeziumSourceConnector`	CDC from PostgreSQL, MySQL, MongoDB
`JdbcSourceConnector`	Polling queries from relational databases
`S3SinkConnector`	Flush to S3-compatible object stores
`ElasticsearchSinkConnector`	Index to Elasticsearch
`HttpSinkConnector`	POST to REST APIs
`JdbcSinkConnector`	Write to relational databases
`BigQuerySinkConnector`	Load into Google BigQuery

12.7 Limitations

- Connector configs are generated as stubs. The user must fill in connector-specific settings (connection URLs, credentials, table names, bucket names).
- The scaffolder does not validate connector configs against a live Connect instance.
- Single Message Transform (SMT) chains are not generated. The user adds transforms in the connector config as needed.

13 Generated Code Extension Strategy

Generated code is a starting point. The BPMN model is the primary design source; generated code is scaffolding expected to be completed and adapted. Durga provides three strategies for extending generated code, ordered by preference.

13.1 Strategy 1: Use a registered plugin

For common data pipeline operations, use one of the 15 registered plugins. Annotate a `ServiceTask` with `camunda:plugin` and `camunda:pluginConfig` in the BPMN model. The scaffolder generates a complete, working worker that delegates to the plugin at runtime. No new code is required.

This covers: field remapping, filtering, type coercion, string templating, PII masking, regex extraction, JSON flatten/unflatten, UUID injection, timestamp normalization, schema validation, key-value enrichment, field-based routing, window counting, and Kafka Connect source/sink configuration.

13.2 Strategy 2: Implement a custom activity

When no plugin covers the required logic, mark the BPMN activity with `camunda:plugin="custom"`. The scaffolder generates:

- A **contract interface** (e.g. `MyTransformContract extends Plugin`) that defines the API between generated and hand-written code.
- A **delegating worker** that injects the contract via CDI (`@Inject`), delegates each message to `implementation.` and includes a null guard with dead-letter routing.

The developer writes an implementation class in a standard Java source file, compiles and tests it with the rest of the project. The generated contract interface and delegating worker are not modified by hand.

A build-time utility (`ModelEnricher`) scans compiled classes for implementations of generated contracts, matches them to BPMN activities, and writes `customImpl`, `customSource`, and `customHash` properties back into the local BPMN model copy. On regeneration, the enriched model carries the implementation references forward — the scaffolder regenerates the contract and worker but never touches the hand-written implementation class.

The BPMN model is copied into `src/main/resources/` in every generated project. It is version-controlled alongside the custom implementation and contract interface. Plugin-only workers can be `.gitignored` since they are fully regenerated from the model.

13.3 Strategy 3: Modify generated code directly

For one-off customizations that don't fit the plugin or custom activity models, modify the generated Java files directly. The scaffolder skips files that already exist in `src/main/java/` on regeneration, so modifications survive.

This approach is reserved for cases where strategies 1 and 2 are insufficient. Custom activity contracts separate generated code from hand-written logic, so regeneration does not overwrite custom work.

13.4 What is typically edited

Common edits in generated projects:

- custom activity implementations (implementing generated contracts),
- gateway condition expressions in the BPMN model (evaluated at runtime by the recursive descent parser — no code modification needed),
- payload handling and validation,
- integration calls inside custom implementations,
- operational configuration values in `application.yml`.

13.5 What should remain generated

Keep these close to the generated structure:

- topic and channel topology,
- lifecycle event emission shape,
- probe script set,
- subprocess and boundary wiring,
- monitoring-facing event semantics.

13.6 Regeneration strategy

When the pipeline needs structural changes (add/remove/reorder activities, change gateway logic, adjust plugin configuration), edit the BPMN model in Camunda Modeler and regenerate the project. For custom activities, start from the enriched model that already carries implementation metadata.

Plugin workers are regenerated from scratch. Contract interfaces are regenerated if the activity configuration changed. Hand-written implementation classes are never touched. If an implementation class was removed or renamed, the scaffolder emits a warning but does not delete the orphaned file.

14 Plugin Architecture

Durga supports a plugin-driven data pipeline model where BPMN `ServiceTask` elements can be annotated with `camunda:plugin` attributes that resolve to registered, governed data processing components.

14.1 Plugin registry

Plugins are defined by YAML descriptors under `plugins/` and indexed by `plugins/catalog.yml`. Each descriptor declares:

- `id` — unique plugin identifier (used in `camunda:plugin`),
- `name` / `version` — human-readable identifier,
- `category` — governed taxonomy (transform, validate, enrich, route, aggregate, connect),
- `status` — lifecycle state (experimental, stable, deprecated, retired),
- `implementation` — Java class implementing `org.gautelis.durga.plugins.Plugin`,
- `config` — typed configuration schema (field types, defaults, allowed values).

14.2 Governed categories

Each plugin category enforces a contract: topic naming, lifecycle events, and error channel conventions. The scaffolder validates that generated executors follow the category contract.

Category	Contract
transform	1:1 input→output, ACTIVITY_COMPLETED, dlq on failure
validate	Output + invalid branch, ACTIVITY_ESCALATED on invalid
enrich	Input→enriched output, may block on external lookup
route	Input→N downstream channels, GATEWAY_TAKEN lifecycle
aggregate	Windowed counting, ACTIVITY_ENTERED on window start
connect	Kafka Connect source/sink configuration generation

14.3 Plugin interface

All plugins implement `org.gautelis.durga.plugins.Plugin`:

```
Stringexecute(Stringpayload,Stringconfig)throwsException
```

Generated executor classes instantiate the plugin at field level and call `plugin.execute(payload, config)` for each message. The config string comes from the BPMN `camunda:pluginConfig` property.

14.4 Plugin catalog

The repository ships with 15 plugins organized by category. Stable plugins are fully supported; experimental plugins generate with a warning.

14.4.1 Transform plugins (9)

JSON Transform Dot-notation field remapping with colon-syntax renaming and literal injection.
Field Filter Whitelist/blacklist field filtering with optional flatten.
Type Coercion Coerces field values between string, int, long, double, decimal, and boolean.
String Template Renders a template string with `${field}` substitution. Dot-notation access for nested fields.
PII Mask Masks sensitive text fields with configurable mask character and boundary character preservation.
Regex Extract Extracts named capture groups from a source field into the payload. Supports multiple matches.
JSON Flatten Flattens nested JSON to dot-notation keys, or unflattens dot-notation back to nested objects. Configurable separator and max depth.
UUID Inject Injects UUIDs (v4 or time-based v1) into specified payload fields.
Timestamp Normalize Converts between epoch_s, epoch_ms, ISO 8601, RFC 3339, and custom DateTimeFormatter patterns. Configurable timezone.

14.4.2 Validate plugins (1)

JSON Schema Validator Validates payloads against a JSON Schema subset. Routes invalid messages to DLQ, skipped, or fails. Status: experimental.

14.4.3 Enrich plugins (1)

KV Enricher Looks up a key field value in an inline data map and shallow-merges enrichment data. Status: experimental.

14.4.4 Route plugins (1)

Field Router Routes messages to named output channels based on a field value. Status: experimental.

14.4.5 Aggregate plugins (1)

Window Counter Counts messages within a tumbling time window and emits a summary record on window close. Optional group-by field. Status: experimental.

14.4.6 Connect plugins (2)

Kafka Connect Source Generates Kafka Connect source connector configuration for pipeline intake topics.
Kafka Connect Sink Generates Kafka Connect sink connector configuration for pipeline egress.

14.5 BPMN binding

Plugin-annotated tasks use Camunda extension properties on `ServiceTask` elements:

```
<bpmn:serviceTask id="transform" name="Transform Data">
  <bpmn:extensionElements>
    <camunda:properties>
      <camunda:property name="plugin" value="json-transform" />
      <camunda:property name="pluginConfig" value="name, email" />
    </camunda:properties>
  </bpmn:extensionElements>
</bpmn:serviceTask>
```

Without a `camunda:plugin` attribute, `ServiceTask` elements fall back to the existing generic worker generation.

14.6 Generated executor

The scaffolder resolves the plugin descriptor, validates it against the registry, and generates a concrete executor class using the `pluginExecutorClass` template. The executor:

- imports the plugin implementation class and the `Plugin` interface,
- instantiates the plugin as a private field,
- delegates each incoming message to `plugin.execute()`,
- emits `ACTIVITY_COMPLETED` on success,
- routes failures to a dead-letter topic (`%task%_dlq`) with structured error records,
- respects `ScopeCancellationRegistry` for interrupting boundary support.

14.7 Custom activities

When no registered plugin covers a transformation, the `camunda:plugin="custom"` marker triggers a different code generation path designed for hand-written implementations:

```
<bpmn:serviceTask id="custom_transform" name="Custom Transform">
  <bpmn:extensionElements>
    <camunda:properties>
      <camunda:property name="plugin" value="custom" />
      <camunda:property name="pluginConfig" value="org.example.CustomTransformCor
    </camunda:properties>
  </bpmn:extensionElements>
</bpmn:serviceTask>
```

Instead of generating a self-contained worker, the scaffolder produces:

1. A **contract interface** (`CustomTransformContract`) extending `Plugin` — this defines the API between generated and hand-written code.
2. A **delegating worker** (`CustomTransformWorkerService`) that injects the contract via CDI (`@Inject`) and delegates each message to `implementation.execute()`. The worker includes a null guard that routes to the dead-letter queue if no implementation is present.

The developer writes an implementation class (`CustomTransformImpl implements CustomTransformCont`) in a standard Java source file. It is compiled and tested with the rest of the project. The generated contract and worker are not modified by hand.

14.7.1 Round-trip model enrichment

A build-time utility (`ModelEnricher`) scans compiled classes for implementations of generated contracts, matches them to BPMN activities by contract name, and writes `customImpl`, `customSource`, and `customHash` Camunda extension properties back into the local copy of the BPMN model. Detection uses a `URLClassLoader` and reflection (`isAssignableFrom`); hashing uses the source file (`.java`) rather than the bytecode (`.class`), so the hash is stable across JDK versions and compiler flags. The source directory is derived from the classes directory by Maven convention (`target/classes` → `src/main/java`) or passed explicitly.

The enriched model becomes the source of truth for future regenerations. When the pipeline changes, regeneration starts from the enriched model — it already knows which activities have custom implementations and which Java class backs each one. The custom implementation files are never touched by the scaffolder.

14.8 Current limitations

- Plugin descriptors are read from the filesystem or classpath at scaffold time — there is no runtime plugin discovery or hot-reload.
- Config validation at scaffold time is not yet implemented — invalid configs are caught at runtime by the generated executor.
- The plugin catalog must be maintained manually; there is no registry publishing or version-resolution tooling.

15 Generated Runtime Classes

Each generated project contains Java classes under Operational classes (workers, gateways, orchestrator) are generated under: `src/main/java/org/gautelis/durga/generated/`. Probes and exercisers (starter, scenario runner, publishers, watchers) are generated under: `src/main/java/org/gautelis/durga/generated/probes/`. These classes map BPMN constructs onto Kafka consumers, producers and lifecycle events rather than hiding behind an opaque engine.

15.1 Class families

- **Starter classes** publish the initial lifecycle event and first task input.
- **Task services** handle BPMN tasks. Service-task handlers auto-complete after their work step; user and manual task handlers enter a waiting state and await explicit completion, failure or escalation input.
- **Plugin executors** instantiate a registered plugin at field level and delegate each message to `plugin.execute()`. No per-plugin code paths in the generator.
- **Custom contract interfaces** define the API for hand-written implementations of custom activities. They extend the `Plugin` interface.
- **Custom delegating workers** inject a contract interface via CDI and delegate each message to the implementation. Includes a null guard with dead-letter routing.
- **Gateway services** handle XOR, AND and OR routing decisions, including fan-out and join behavior. XOR and OR conditions are evaluated at runtime by a recursive descent expression parser supporting `==`, `!=`, `>`, `<`, `>=`, `<=`, `&&`, `||`, `!`, parentheses, and nested field access.
- **Event handlers** implement timer, message and signal behavior for intermediate events and boundary-event reactions.
- **Call-activity helpers** model request and reply behavior for call activities.
- **Subprocess scope services** emit subprocess entry and completion lifecycle events and coordinate subprocess-local routing.
- **Process orchestrator** observes terminal flows and emits process completion.
- **Probe helpers** provide generated publishers and watchers for command-line exploration.

15.2 Task services

Generated task services consume a task input topic, emit `ACTIVITY_ENTERED`, perform the generated action or enter a wait state, then emit `ACTIVITY_COMPLETED`, `FAILED`, `ESCALATED` or `CANCELLED` before forwarding to the next topic or boundary path.

15.3 Plugin executors and custom workers

Plugin executors and custom delegating workers share the same structure: a CDI bean with `@Incoming` for the task input topic and `@Channel` emitters for output, process-events, and dead-letter topics. The difference is in how the transformation logic is obtained:

- **Plugin executor**: instantiates the concrete plugin class as a field (`private final Plugin plugin = new JsonTransform();`) and calls `plugin.execute(payload, config)`.

- **Custom delegating worker**: injects the contract interface via CDI (`@Inject MyContract implementatio` and calls `implementation.execute(payload, config)`. The implementation is provided by a developer-written class annotated with `@ApplicationScoped`.

Both patterns include scope cancellation checks, DLQ routing on failure, and canonical `process-events` emission.

15.4 Routing and event classes

Gateways turn BPMN branching into explicit topic routing. Timer, message and signal handlers make delays and external triggers concrete. Boundary handlers observe task or scope state and branch into exceptional or reminder flows when trigger conditions are met.

15.5 Scope classes

Subprocess scope services emit subprocess-level entered and completed events, mediate flow into and out of the subprocess graph, propagate failure, escalation and cancellation at scope level, and support nested subprocess structure. Event subprocess handlers model subprocess-internal event-driven branches separately from the main task path.

16 Naming Conventions

Durga relies heavily on predictable names because the generated topology is intentionally visible. Naming is therefore part of the system design, not only a cosmetic detail.

16.1 Common naming patterns

The generator generally follows these patterns:

- process identifiers come from BPMN process ids,
- task topics use process and task identifiers,
- generated classes use process and BPMN element names converted to Java class names,
- helper scripts use short imperative file names such as `complete-task.sh`,
- monitoring topics use shared names such as:

```
process-events
process-state
process-state-counts
```

16.2 Why this matters

Consistent names make it easier to:

- inspect topic traffic in Kafka tools,
- map BPMN elements to generated Java handlers,
- reason about subprocess and boundary scope behavior,
- debug routing and completion behavior quickly.

16.3 Practical guidance

For the cleanest generated output, BPMN ids should be stable, readable and explicit. If BPMN models use unclear or heavily tool-generated ids, the generated Kafka and Java output becomes harder to read as well.

If the BPMN tool changes element identifiers between edits, pin the process id with `-process-id <id>` to keep topic names, consumer groups, and class names stable across regenerations.

17 Generated Project Lifecycle

Generated projects are used iteratively, not as one-off code dumps:

1. model a BPMN process,
2. scaffold a project from that model,
3. inspect generated topics, classes and helper scripts,
4. run demo flows or manual probes,
5. observe the lifecycle stream and monitoring views,
6. revise the BPMN model and regenerate.

17.1 Two modes of use

- as a learning scaffold, where the value is seeing how BPMN maps onto Kafka,
- as a starting point for a real implementation, where the generated code becomes a base that is extended, completed and hardened.

The BPMN model remains the primary design artifact. Generated output is readable, runnable and observable by design.

18 Demo Flows And Helper Scripts

18.1 Purpose

Generated scripts provide runnable examples of the Kafka topology, reduce the amount of manual producer and consumer work needed during exploration, and make BPMN semantics visible through actions such as completion, failure, escalation and observation.

18.2 Common generated scripts

The exact set depends on the BPMN model. Common script categories include:

- `demo-scenario.sh` for whole-process happy, stuck or failed runs,
- `send-task-input.sh` for manual injection into one task input topic,
- `complete-task.sh`, `fail-task.sh` and `escalate-task.sh` for human or exceptional task outcomes,
- `complete-call-activity.sh` for returning from a generated call activity,
- `send-message-event.sh` and `send-signal-event.sh` for external event stimulation,
- `watch-process-events.sh` and `watch-task-output.sh` for runtime observation.

18.3 Recommended learning loop

1. generate a project from one small BPMN sample,
2. create the Kafka topics and start the generated runtime,
3. run `watch-process-events.sh`,
4. execute `demo-scenario.sh happy`,
5. repeat with a failure, escalation or timeout-oriented script,
6. inspect the monitoring endpoints or dashboard after each run.

This loop exposes the difference between nominal and exceptional flow with very little setup. The execution topics carry work, but the canonical `process-events` stream carries the lifecycle truth.

19 Configuration And Operations

19.1 Scaffolder invocation

The repository includes a convenience script at `durga`:

```
./durga path/to/process.bpmn [flags...]
```

It auto-builds the JAR if missing and passes all arguments through to the scaffolder. Equivalent to:

```
mvn -q package -DskipTests
java -jar target/durga-0.1.0-beta.1.jar path/to/process.bpmn [flags...]
```

Available flags: `-dry-run`, `-out <dir>`, `-transactions`, `-separate-workers`, `-strimzi`, `-connect`, `-event-topic <topic>`, `-process-id <id>`, `-package <pkg>`, `-retention <hours>`.

`-retention <hours>` sets Kafka topic retention in hours (defaults to 168 h/7 days with a warning). Use `-1` for infinite retention. The canonical event topic (configured via `-event-topic`) always has infinite retention; the `%process%_state` topic always uses compaction. Wired into both `topics.sh` and Strimzi `topics.yml`. Accepts `h`, `d`, `w`, `m` suffixes for hours, days, weeks, or 30-day months.

For production, do not accept generated retention defaults without review. Retention is part of the process contract: it controls replay windows, diagnostic history, storage cost and monitoring recovery. Infinite retention requires an explicit storage and compaction strategy.

`-package <pkg>` sets the Java package for generated code (defaults to `org.gautelis.durga.generated` with a warning). Operational classes are placed directly in the package; probe and exerciser classes in `<pkg>.probes`.

`-event-topic <topic>` overrides the physical Kafka topic name for canonical lifecycle events. The default is `process-events-<processId>`, so each pipeline writes to its own topic by default. Use this flag to direct multiple pipelines to a shared topic (e.g. `-event-topic process-events`) or to use a custom naming scheme. The logical channel names `process-events` and `process-events-monitor` in the generated code stay unchanged; only the physical topic mapping in `application.yml` is affected.

`-process-id <id>` overrides the process identifier read from the BPMN file. This is the name used in topic names, consumer group IDs, generated class names, and the `durga-<id>` consumer group prefix for the `ProcessStateStore`. Pinning the process ID keeps Kafka group coordination stable if the BPMN tool regenerates with different element identifiers.

19.2 Local runtime baseline

The normal local environment consists of:

- the bundled Kafka setup under `setup/`,
- generated topic configuration in the scaffolded project,
- the optional monitoring app and dashboard.

The repository is intentionally optimized for a local single-broker loop so the generated processes and monitoring projections can be explored quickly. This local baseline is not a production topology; production deployments need broker replication, authenticated clients, managed secrets, resource limits, topic ownership and backup/replay policy.

19.3 Important configuration surfaces

The main configuration locations are:

- repository-level runtime defaults in `src/main/resources/application.yml`,
- generated-project channel configuration in the scaffolded `application.yml`,
- local Kafka infrastructure in `setup/docker-compose.yml`,
- generated topic setup scripts in scaffolded output directories.

19.4 Operational expectations

In normal local startup:

- Kafka UI may log temporary connection failures before the broker is ready,
- generated runtimes rely on explicit topics and channels rather than implicit discovery,
- the monitoring app may report a transitional Streams state before its stores become queryable.

19.5 Where state lives

Durga uses several layers of state:

- transport history on execution topics,
- canonical lifecycle history on `process-events`,
- materialized latest state and counts in Kafka Streams stores and corresponding topics,
- local waiting and cancellation bookkeeping inside some generated handlers.

Operationally, the first three layers are Kafka-backed and can be recovered or replayed within their configured retention windows. The last layer is process memory and should be treated as transient unless the generated handler explicitly persists its coordination state.

19.6 Recommended local development loop

For practical local work:

1. start Kafka,
2. build the repository or generated project,
3. create topics,
4. run the generated process or a demo publisher,
5. start the monitoring app if query visibility is needed,
6. inspect lifecycle events, counts and latency.

20 Testing And Verification

20.1 Generator verification

Three layers:

- dry-run tests that treat scaffold summaries and previews as a contract,
- generation tests that write projects to temp directories and verify output (file existence, plugin imports, condition expressions, contract interfaces),
- a compilation integration test that generates a full project from `invoice_receipt.bpmn` and compiles all `.java` files with `javax.tools.JavaCompiler`, verifying the generated code is syntactically and semantically correct.

The verification set spans 24 BPMN fixtures covering:

- message and signal events,
- timers and boundaries,
- subprocesses and nested subprocesses,
- call activities,
- event subprocesses (message, signal, timer, error, escalation),
- data pipelines (plugin-annotated tasks, custom activities),
- gateway condition expressions.

20.2 Plugin verification

Each of the 13 plugin implementations has dedicated unit tests (55 total) covering normal operation, edge cases, and error handling. The plugin registry loading and descriptor validation are exercised by the scaffolder tests.

20.3 Data pipeline verification

Four dedicated tests verify:

- the plugin-annotated pipeline (`data_pipeline_demo.bpmn`) generates correct plugin executor classes,
- the order events pipeline (`order_events_pipeline.bpmn`) generates workers for 8 plugin types and an XOR gateway with runtime-evaluated conditions,
- the log processing pipeline (`log_processing_pipeline.bpmn`) generates workers for regex extraction, templating, flattening, and validation plugins,
- the custom activity demo (`custom_activity_demo.bpmn`) generates contract interfaces and delegating workers.

20.4 Connect verification

Two tests verify that `-connect` generates source and sink connector configs with correct topic lists for both dry-run and full generation modes.

20.5 Model enricher verification

Three tests verify the `ModelEnricher`: enrichment of custom activity metadata, no-op when no implementation is found, and property update on re-enrichment. Tests compile real Java source files during execution and verify that `customSource` references the `.java` path (not the `.class` file) and that hashes are computed from source files.

20.6 Core runtime contract verification

- `ProcessEvent` serialization round-trips and compact-constructor inference,
- `ProcessState` serialization round-trips,
- `ScopeCancellationRegistry` cancellation tracking, merging, clearing and null-guard behavior,
- `ProcessStateView` projection tests covering cancellation, escalation, process completion, gateway events, concurrent activities, and state-key generation.

20.7 Monitoring verification

- activity latency statistics (min, max, avg, p50, p95, p99, SLA violations),
- demo publishers and scenario runners that drive the monitoring topology without a full generated process runtime.

20.8 CI pipeline

GitHub Actions runs `mvn test` on push and PR to `main` using JDK 21. 196 tests execute in approximately 25 seconds.

20.9 Not fully covered

- full end-to-end process executions against a live Kafka broker,
- load and concurrency behavior,
- multi-instance monitoring scenarios.

21 Troubleshooting

Most operational failures are caused by startup timing, missing topics, incomplete task logic, Kafka connectivity, or confusion between execution state and monitoring state. Start with the runtime signals before assuming a generator defect.

21.1 Kafka UI logs connection failures at startup

This is usually normal in the local setup. Kafka UI often starts before the broker is fully ready and logs transient connection failures. If the broker then comes up and the errors stop, this is not itself evidence of a Durga runtime failure.

21.2 Monitoring app says the state store is not queryable yet

This normally means Kafka Streams has started but the state stores are not yet ready for queries. The correct response is usually to wait briefly and retry rather than to assume the topology is broken immediately.

In production, repeated or long-lived readiness failures should be treated as an operational incident. Check broker reachability, lifecycle-topic consumer lag, Streams application id stability, changelog topic health and whether a deployment started with a fresh state directory or missing permissions.

21.3 A generated process compiles but does not advance

The most common causes are:

- topics were not created,
- the relevant generated application is not running,
- an XOR branch condition still contains placeholder logic,
- a user or manual task entered a waiting state and needs explicit completion,
- the process instance was cancelled by a boundary or interrupting event subprocess,
- the worker processed a record but failed before committing its offset, causing a replay that requires idempotent handler logic.

21.4 Boundary behavior does not appear to trigger

Check these points first:

- the triggering lifecycle event is actually emitted,
- the boundary is interrupting or non-interrupting as intended,
- the attached task or subprocess reached the expected state,
- failure or escalation was emitted through the generated helper path rather than by bypassing the lifecycle model.

21.5 An external completion arrives after cancellation

The generated runtime now suppresses much of this post-cancel progress, but late events can still be confusing during debugging. In such cases, the canonical `process-events` stream is the best source of truth: check whether cancellation happened before the late completion arrived.

21.6 Monitoring counts do not match raw topic intuition

This is usually a model misunderstanding rather than a bug. Execution topics contain history and routing traffic. The monitoring counts are derived from the latest projected instance state, which is the intended read model for "how many instances are in state X right now".

21.7 Dashboard trend or latency data looks incomplete

The dashboard reads materialized monitoring stores, not raw execution topics. Missing trend or latency rows usually means no supported lifecycle event has reached the monitoring topology yet, the Streams stores are still restoring, or old monitoring topics contain records produced by an older projection version. For persistent inconsistencies, compare `process-events`, `process-trends`, `process-latency`, and the monitoring application logs.

21.8 The generated output looks harder to understand than expected

This often comes from BPMN naming. Unclear BPMN ids produce unclear Java classes, topics and helper artifacts. Cleaning up the BPMN model identifiers usually improves the generated output more than post-processing the generated files.

22 Known Design Tensions

22.1 Explicitness versus compactness

The generated runtime is larger than a hidden workflow runtime would be. This is accepted because the BPMN-to-Kafka mapping is visible, making the system easier to inspect and debug.

22.2 BPMN fidelity versus pragmatic Kafka mapping

BPMN semantics are projected through explicit Kafka handlers and topics rather than a full engine. Some semantics are intentionally pragmatic rather than exhaustive.

22.3 Regeneration versus local customization

Generated projects invite local edits, but the BPMN model remains the primary artifact. When the model changes substantially, regeneration with re-application of focused logic changes is preferred over manual rewiring of the generated topology.

22.4 Exploration versus production hardening

The system is strong enough for architectural exploration, but not every part is optimized for hardened production operation. The current focus remains clarity, runnable demos, and generator correctness.

23 BPMN Sample Catalog

The repository contains a growing set of BPMN models under `src/test/resources/bpmn`. They serve two purposes:

- they are regression inputs for the scaffolder and its tests,
- they are concrete learning models for understanding one BPMN feature at a time.

The samples are intentionally small. Most of them isolate one feature rather than trying to model a realistic end-to-end business process in full detail.

Model	Primary focus
`invoice_receipt.bpmn`	Baseline process with start, service work, review, approval or rejection, and terminal notification. This is the best first model to inspect.
`order_fulfillment.bpmn`	Legacy process sample kept as an additional reference model. It is useful for checking that generator changes are not too tightly coupled to the invoice-oriented examples.
`invoice_receipt_reminder.bpmn`	Intermediate timer catch event between normal process steps. Demonstrates delay-driven routing without a boundary event.
`invoice_message_exchange.bpmn`	Intermediate message throw and catch events. Shows explicit Kafka topic mappings for message-based collaboration.
`invoice_signal_exchange.bpmn`	Intermediate signal throw and catch events. Useful for contrasting signal semantics with the more direct message-event mapping.
`invoice_review_deadline.bpmn`	Interrupting timer boundary on a task. Demonstrates timeout handling and normal-flow suppression after deadline expiry.
`invoice_review_reminder_non_interrupting.bpmn`	Non-interrupting timer boundary on a task. Demonstrates reminder-style branching without cancelling the main task.
`invoice_processing_error.bpmn`	Interrupting error boundary on a task, driven by task failure events from the generated runtime.
`invoice_review_escalation.bpmn`	Interrupting escalation boundary on a task. Shows how escalation differs from failure as an explicit lifecycle outcome.
`invoice_call_activity.bpmn`	Call-activity request and reply mapping. Demonstrates the generated call and reply topics plus manual completion of the invoked activity.
`invoice_review_subprocess.bpmn`	Embedded subprocess with explicit entry and completion handlers. This is the basic scope-aware subprocess sample.
`invoice_nested_subprocess.bpmn`	Nested embedded subprocesses. Useful for understanding recursive scope generation and subprocess-specific input and output channels.

Model	Primary focus
`invoice_subprocess_deadline.bpmn`	Interrupting timer boundary attached to a subprocess scope. Demonstrates cancellation at the subprocess level rather than at one task.
`invoice_subprocess_reminder_non_interrupting.bpmn`	Non-interrupting timer boundary attached to a subprocess scope. Shows branch-without-cancel behavior for a whole scope.
`invoice_subprocess_error.bpmn`	Interrupting error boundary attached to a subprocess scope. Demonstrates subprocess-level failure propagation.
`invoice_event_subprocess_message.bpmn`	Non-interrupting embedded event subprocess with message start. Shows side-flow activation within an enclosing subprocess scope.
`invoice_event_subprocess_interrupting_message.bpmn`	Interrupting embedded event subprocess with message start. Demonstrates cancellation of the enclosing scope before the event subprocess branch starts.
`invoice_event_subprocess_timer.bpmn`	Embedded event subprocess with timer start. Demonstrates scoped delayed activation tied to the enclosing subprocess lifecycle.
`invoice_event_subprocess_error.bpmn`	Embedded event subprocess with error start. Demonstrates start-on-failure semantics driven by the canonical lifecycle stream.
`invoice_event_subprocess_escalation.bpmn`	Embedded event subprocess with escalation start. Demonstrates start-on-escalation semantics within a subprocess scope.
`data_pipeline_demo.bpmn`	Minimal data pipeline with three plugin-annotated tasks: JSON Transform, Field Filter, and KV Enricher. The simplest plugin catalog example.
`order_events_pipeline.bpmn`	Realistic order event pipeline with 8 plugin tasks (transform, coerce, filter, enrich, validate, UUID-inject, timestamp-normalize, PII-mask) and an XOR gateway routing by amount using runtime-evaluated conditions. Designed to be used with `-connect` for source/sink generation.
`log_processing_pipeline.bpmn`	Log processing pipeline demonstrating regex extraction, type coercion, string templating, JSON flattening, schema validation, and PII masking in a single linear flow. Also designed for `-connect` source/sink integration.
`custom_activity_demo.bpmn`	Custom activity round-trip example. A ServiceTask annotated with `camunda:plugin="custom"` triggers contract interface generation and delegating worker creation for hand-written implementations.

Taken together, the sample set functions as a compact executable coverage matrix. When a new BPMN function is added to the generator, adding a dedicated sample model alongside it has become the preferred way to document the feature, test the generator, and provide a runnable teaching example.

24 Example Walkthrough

24.1 Model

The reference model is `src/test/resources/bpmn/invoice_receipt.bpmn`: start, service work, review, approve/reject, notify.

24.2 Generation

```
./durga src/test/resources/bpmn/invoice_receipt.bpmn
```

Or directly:

```
mvn -q clean package
java -jar target/durga-0.1.0-beta.1.jar \
  src/test/resources/bpmn/invoice_receipt.bpmn
```

Output lands in `generated/` with Java sources, `pom.xml`, Kafka topic declarations, helper scripts, and a generated `README.md`.

24.3 Generated contents

- worker services for automatic task execution,
- waiting-state handlers for human-completion steps,
- a starter class that publishes the first process event,
- a process orchestrator that detects terminal completions,
- helper publishers and watchers for demos and testing,
- Kafka bindings in `application.yml`.

24.4 Runtime flow

1. the starter publishes the initial process event,
2. task workers consume input topics,
3. each step emits lifecycle events to `process-events`,
4. routing services move the process to the next task or end event,
5. the orchestrator emits `PROCESS_COMPLETED`.

Richer models add boundaries, subprocess scopes, and event subprocesses but follow the same pattern: handlers move through Kafka topics while `process-events` carries the lifecycle truth.

24.5 Exploring the generated process

- `demo-scenario.sh` exercises a whole process path,

- `send-task-input.sh` pushes a specific task input,
- `complete-task.sh`, `fail-task.sh`, `escalate-task.sh` force specific task outcomes,
- `watch-process-events.sh`, `watch-task-output.sh` expose the runtime flow.

25 Current Status

25.1 Maturity

Active prototype with beta-oriented hardening work underway. The generator covers the BPMN execution subset described in the execution model chapter and is regression-tested against the BPMN fixtures under `src/test/resources/bpmn/`. Generated projects compile and run on Quarkus + Kafka, but production adoption still requires the operational review described in the deployment, supervision and configuration chapters.

25.2 Solid

- generator refactored from monolith to multiple templates and helper classes,
- dry-run and compile-level tests cover representative BPMN features,
- core runtime contracts are unit-tested,
- monitoring topology with global-table query model,
- interrupting boundary cancellation via in-memory registry,
- GitHub Actions CI on push/PR (JDK 21 plus monitoring UI test/build gate),
- `SendTask`, `ReceiveTask`, `ScriptTask`, `BusinessRuleTask` generate dedicated handlers,
- inclusive gateway (OR) split/join,
- multi-instance loop characteristics detected and reported,
- 15 data pipeline plugins (9 transform, 1 validate, 1 enrich, 1 route, 1 aggregate, 2 connect) with interface-based dispatch,
- recursive descent expression evaluator for XOR and OR gateway conditions (==, !=, >, <, >=, <=, &&, ||, !, parentheses, nested field access),
- custom activity round-trip: contract interface generation, delegating workers with CDI injection, model enrichment utility.

25.3 Limits

- BPMN subset (no complex gateways, no data objects),
- advanced scope semantics approximated,
- event subprocess semantics scoped, not generalized to all top-level cases,
- history retention is intentionally coarse,
- production deployment requires site-specific review of topic retention, replication, authentication, secrets, resource limits and alerting.

25.4 Open decisions

Top-level event subprocesses Timer, error, and escalation starts are out of the beta support boundary unless promoted with dedicated runtime semantics and integration coverage.

Runtime fidelity Whether to deepen true BPMN scope semantics as the project evolves.

Multi-instance runtime Whether detected multi-instance loop characteristics should be promoted from summary metadata to generated runtime handlers after beta.

Plugin SDK Whether to formalize a plugin development kit for third-party plugin authors beyond the current `Plugin` interface contract.

26 Glossary

BPMN Business Process Model and Notation, the process-modeling notation used as input to Durga.

Canonical lifecycle stream The shared `process-events` topic that carries normalized process and activity events.

Call activity A BPMN element used to model invocation of another process-like unit, mapped in Durga through explicit call and reply topics.

Event subprocess A subprocess triggered by an event within an enclosing scope instead of by the main sequence flow.

Execution topics The generated Kafka topics that move work between tasks, gateways and handlers.

Generated helper scripts The scaffolded scripts used to run demos, inject inputs, complete tasks, force failures and observe output.

Kafka Streams state store The local materialized store used by the monitoring app to answer low-latency queries.

Process orchestrator The generated runtime class that detects terminal flows and emits explicit completion for the enclosing process.

ProcessStateView The repository's materialized per-instance monitoring projection.

Scope cancellation The generated runtime behavior that suppresses or redirects normal flow after an interrupting boundary or interrupting event subprocess fires.

Scaffolder The BPMN-driven generator entry point, implemented by:

```
org.gautelis.durga.tools.BpmnScaffolder
```

Subprocess scope service A generated class that manages entry, completion and scope-level lifecycle behavior for an embedded subprocess.